

Rapport de Soutenance



Gaël KERNINON
Thibault Bichet
Baptiste Vrbnac
Ethan Rouimi

Projet OCR

4 novembre 2025

Contents

I	Informations Générales	2
1	Informations Générales	3
1.1	Informations sur le Projet	3
1.1.1	Résumé	3
1.1.2	Membres	3
1.2	Informations sur l'Implémentation	4
1.2.1	Outils et Logiciels	4
1.2.2	Architecture	4
II	Rapports Hebdomadaires	5
2	Rapports Hebdomadaires	6
2.1	Semaine du 22 septembre 2025	6
2.2	Semaine du 29 septembre 2025	7
2.3	Semaine du 6 octobre 2025	10
2.4	Semaine du 13 octobre 2025	13
2.5	Semaine du 20 octobre 2025	15
2.5.1	Code behind et débogage GTK	16
2.6	Semaine du 27 octobre 2025	17
2.6.1	Détection caractères (Thibault)	17
III	Bibliographie et Sources	19
3	Bibliographie et Sources	20
3.1	Technique	20
3.1.1	Structures de Données	20
3.1.2	Algorithmes	20
3.1.3	Tutoriels et Ressources Vidéo	20
3.1.4	Librairies	20

Part I

Informations Générales

Chapter 1

Informations Générales

Introduction

Le projet **OCR KeltIA** s'inscrit dans le cadre du module de projet du semestre 3 à l'*EPITA*. Réalisé en équipe de quatre étudiants, il a pour objectif la conception et l'implémentation d'un logiciel de reconnaissance optique de caractères (OCR) capable d'analyser et de résoudre automatiquement une grille de mots cachés à partir d'une image. Ce projet associe plusieurs domaines fondamentaux de l'informatique : le traitement d'image, l'intelligence artificielle, la reconnaissance de formes et le développement bas-niveau en langage C.

Notre démarche s'articule autour de la rigueur, de la modularité et de la performance, en respectant les contraintes imposées par le cahier des charges du projet **OCR Word Search Solver**.

1.1 Informations sur le Projet

1.1.1 Résumé

Le projet vise à développer un système complet de reconnaissance de texte à partir d'images, intégrant des modules de prétraitement, de segmentation, de reconnaissance de caractères et de résolution automatique d'une grille de mots cachés. L'application, écrite en langage C, repose sur une architecture modulaire composée de plusieurs sous-systèmes : détection des zones de texte, découpage des caractères, entraînement et inférence d'un réseau de neurones artificiels, puis résolution logique de la grille. En sortie, le programme génère une image annotée contenant la grille résolue. Ce projet met en avant des compétences en algorithmique, en programmation système et en apprentissage automatique, tout en respectant les bonnes pratiques de développement collaboratif via *Git*.

1.1.2 Membres

Le groupe est composé des quatre étudiants suivants :

- Gaël Kerninon
- Thibault Bichet
- Ethan Rouimi
- Baptiste Vrbanac

Chaque membre participe activement à la conception, à l'implémentation et à la documentation du projet, en se répartissant les tâches selon les domaines de compétence de chacun. Les détails de ces tâches sont décrits dans la seconde partie.

1.2 Informations sur l'Implémentation

1.2.1 Outils et Logiciels

Le développement du projet s'effectue principalement sous environnement GNU/Linux, à l'aide d'outils et de logiciels libres :

1. Langage de programmation : C (standard C99)
2. Compilateur : GCC, avec des options comme `-Wall`, `-Wextra`, etc. pour garantir la robustesse du code
3. Environnements de développement : Vim et Visual Studio Code
4. Système de build : Makefile
5. Formatage automatique : `clang-format` via un hook Git pré-commit
6. Gestion de versions : Git (hébergement sur dépôt commun sur GitHub)
7. Rédaction des documents de projet : L^AT_EX

1.2.2 Architecture

1.2.2 Architecture

L'architecture du projet est organisée selon une structure claire, permettant une séparation nette entre les différentes composantes logicielles. Le répertoire principal `KeltIA/` contient notamment :

- `src/` : les fichiers sources organisés par modules¹ ;
- `include/` : les fichiers d'en-tête correspondants.
- `bins/` : les exécutables produits lors de la compilation ;
- `ressources/` : les jeux d'images et les données d'apprentissage nécessaires à la reconnaissance optique ;
- `scripts/` : les scripts d'automatisation et les *Git hooks* utilisés pour le formatage et la gestion du code ;
- `Makefile` : la description des règles de compilation et des dépendances du projet.

L'ensemble est conçu pour permettre une évolution progressive du projet, en facilitant la phase de développement pour les membres du groupe. Cette organisation modulaire garantit la maintenabilité du code, la clarté des dépendances et la cohérence globale du système. De plus, nous avons également ajouté un `README` rédigé en *Markdown* à la source du projet pour aider à se familiariser avec le projet.

¹Traitement d'image, réseau de neurones, résolution de grille, etc.

Part II

Rapports Hebdomadaires

Chapter 2

Rapports Hebdomadaires

2.1 Semaine du 22 septembre 2025

Réunion de Projet (Baptiste, Gaël, Thibault, Ethan)

Le groupe a organisé deux réunions afin de planifier le déroulement du projet et répartir les différentes tâches entre les membres. La première réunion s'est tenue en présentiel, tandis que la seconde a été réalisée en visioconférence. Lors de ces réunions, le planning d'avancement a été défini, et les responsabilités de chacun ont été précisées : Gaël sera en charge des programmes Solver ainsi que de la détection des zones dans une image, Ethan s'occupera du réseau de neurones, tandis que Baptiste prendra en main les traitements d'image tels que la rotation, la conversion en niveaux de gris et la suppression de bruit. Enfin, Thibault développera l'interface utilisateur. Cette organisation a permis d'assurer une répartition claire des responsabilités et de suivre efficacement l'avancement du projet.

Solver (Gaël)

Cette semaine, Gaël a commencé la tâche du solver qui consiste à implémenter un algorithme permettant de trouver un mot dans une grille de lettres. Cet algorithme sera dans un premier temps implémenté en `Python`, puis sera traduit en langage `C`, afin d'assurer une bonne compréhension et réflexion autour de l'algorithme avant sa version définitive dans le projet. L'algorithme principal est encapsulé dans la fonction : `solver(grid, word)`. On itère la grille de lettres à la recherche du mot, pour chaque lettre, et pour chaque direction¹, les lettres de la grille sont comparées successivement à celles du mot, si toutes les lettres correspondent, la fonction retourne les coordonnées de début et de fin du mot, et renvoie `None` sinon. L'algorithme s'arrête automatiquement dès qu'un caractère ne correspond pas ou que les limites de la grille sont dépassées. Ce script inclut également des tests unitaires permettant de vérifier la recherche de mots horizontaux, verticaux et diagonaux, ainsi que la gestion des cas où le mot n'est pas présent, assurant ainsi la fiabilité et la flexibilité du solver pour des grilles de taille modérée. On notera que pour une grille de dimensions $n \times m$ et un mot de longueur l , la complexité de l'algorithme est de l'ordre de $O(n \times m \times l)$. La prochaine étape du projet consistera à implémenter cet algorithme en langage `C`.

À FAIRE :

- Solver (Gaël) : Implémenter le solver en langage `C` et le tester.
- Grayscale (Baptiste) : Implémenter le grayscale en langage `C`.
- Réseau de Neurones (Ethan) : Implémenter le `XNOR` avec un réseau de neurones.
- UI (Thibault) : Trouver la meilleur bibliothèque pour créer des UI

¹Solver : Les 8 directions dans lesquelles le solver cherche un mot sont N, S, E, W, NW, NE, SE, SW

2.2 Semaine du 29 septembre 2025

Grayscale (Baptiste)

La première tâche de Baptiste dans ce projet consistait à implémenter une conversion d'une image en couleur vers une image en niveaux de gris (**Grayscale**). Cette tâche vise à simplifier le traitement des images à l'avenir en supprimant les informations de couleur, afin de ne conserver que les intensités lumineuses des pixels. En résumé, l'objectif de cette première étape est de convertir une image en couleur en une image en noir et blanc.

Dans un premier temps, Baptiste a opté pour l'algorithme suivant : on calcule la moyenne des intensités des pixels de l'image, cette moyenne servant de seuil de binarisation. Chaque pixel dont l'intensité était inférieure ou égale au seuil était alors transformé en noir, et les autres en blanc. L'algorithme initial reposait donc sur le calcul de la moyenne globale de l'image : on calcule la somme des composantes RGB² de tous les pixels, que l'on divise par le nombre total de pixels. Cependant, cette méthode a révélé un problème d'efficacité. Il a donc fallu opter pour une autre méthode.

Par la suite, Baptiste a mis en place un second algorithme, fondé sur une formule intégrant les valeurs minimale (Min_i) et maximale (Max_i) d'intensité de l'image. La formule utilisée est la suivante :

$$Min_i + (Max_i - Min_i) \times 0.1$$

Cette méthode s'est montrée plus adaptée, mais elle demeure peu efficace pour les images présentant une luminosité trop faible ou, au contraire, trop élevée, due à la présence de pixels respectivement noirs ou blancs.

Solver (Gaël)

La semaine suivante, Gaël a effectué l'implémentation du solver en langage C, en conservant la logique de recherche utilisée précédemment en Python. La fonction principale `solver` parcourt chaque case de la grille et explore les huit directions possibles afin de détecter le mot à rechercher. Les coordonnées de la grille sont représentées sous forme d'un tableau linéarisé, ce qui permet de calculer les indices d'accès en utilisant `grid[i * width + j]`. Pour chaque direction, les lettres sont comparées successivement et, si toutes correspondent, les coordonnées de début et de fin du mot sont affichées, sinon, le programme indique que le mot n'a pas été trouvé. Cette implémentation conserve la même complexité algorithmique que la version Python, à savoir $O(n \times m \times l)$ pour une grille de dimensions $n \times m$ et un mot de longueur l , et bénéficie des performances accrues du langage C pour les grilles de taille plus importante ou les traitements en temps réel. Les tests ont été réalisés et vérifient bien les attendus. Cette tâche est désormais considérée comme aboutie et sera merge dans la branche `main` dans les jours suivants.

Outils CI/CD (Ethan)

Cette semaine, Ethan a travaillé sur la mise en place d'outils de CI/CD³ afin de faciliter le développement et la maintenance du projet. Un système de tests automatisé en langage C a été conçu pour vérifier le bon fonctionnement des différents modules à chaque mise à jour du code, garantissant ainsi la stabilité du projet au fil des développements.

²RGB : Abbréviation pour *Red, Green, Blue*, composantes de la couleur d'un pixel en synthèse additive.

³(Intégration Continue et Déploiement Continu)

Réseau de Neurones (Ethan)

En parallèle de l'ajout des Outils CI/CD, Ethan a commencé la conception des premières structures dédiées aux réseaux de neurones, en définissant notamment les structures (**Layer**) et le **Neuronal Network**. Ces structures permettent de représenter les couches d'un réseau et de réaliser les calculs nécessaires à sa propagation. Ethan a également implémenté un mécanisme de sérialisation sous la forme d'un fichier personnalisé au format **.nn**, permettant d'enregistrer et de recharger des modèles de réseaux de neurones. Cette approche assure la persistance des modèles et facilite leur réutilisation ou leur transfert entre différents environnements d'exécution.

Enfin, un premier prototype fonctionnel (POC) de réseau de neurones a été développé pour résoudre le problème logique du **NXOR**⁴, dont la porte est présente ci-contre. en s'appuyant sur des perceptrons. Ce test a permis de valider les principes de base du calcul neuronal et de confirmer le bon fonctionnement des structures et mécanismes de sauvegarde précédemment mis en place.

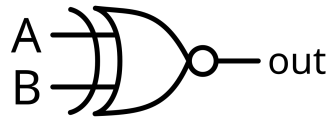


Figure 2.1: Une Porte XNOR

```

Save and Load NNs
[PASS] write nn
[PASS] load nn
[PASS] write nn, load

XNOR NN for demo

Testing XNOR function:
✓ 0 XNOR 0 = 1.00 (expected 1)
✓ 0 XNOR 1 = 0.00 (expected 0)
✓ 1 XNOR 0 = 0.00 (expected 0)
✓ 1 XNOR 1 = 1.00 (expected 1)
[PASS] test XNOR nn

```

Figure 2.2: Résultats du Réseau de Neurones (*perceptrons*) pour la porte XNOR

⁴NXOR ou XNOR : $\overline{A \oplus B} = \overline{A} \cdot \overline{B} + A \cdot B$

Choix de la bibliothèque d'UI

Le premier pas dans la création de l'interface graphique, fût pour Thibault de choisir quel architecture utiliser. Nous avons donc choisi la bibliothèque GTK 3.0. Cette bibliothèque était selon nous meilleur choix car elle est largement supportée, y compris par le PIE, elle permet de créer facilement des interfaces modernes et intuitives à partir d'une bibliothèque de composant. Également, cette bibliothèque permet de s'affranchir des difficulté syntaxiques du C en permettant développeurs à utiliser un fichier XML pour créer l'arbre de composants et leur attribuer des propriétés d'affichage. Nous avons choisi d'utiliser le logiciel Glade pour générer le fichier XML contenant les interfaces.

À FAIRE :

- Grayscale (Baptiste) : Envisager d'utiliser l'algorithme d'Otsu.
- Détection des Zones (Gaël) : Réfléchir à l'algorithme pour détecter les zones d'une image.
- Réseau de Neurones (Ethan) : Continuer les recherches sur le réseau de neurones.
- Créer la structure de l'interface sur glade

2.3 Semaine du 6 octobre 2025

Grayscale (Baptiste)

Pour résoudre le problème de la précédente méthode, inefficace pour les images présentant une luminosité trop faible ou trop élevée, Baptiste a finalement opté pour l'utilisation de l'algorithme d'Otsu. Cet algorithme s'appuie sur l'histogramme des intensités de l'image, c'est-à-dire sur la répartition du nombre de pixels pour chaque niveau de gris. Il permet de déterminer automatiquement le seuil optimal de binarisation en minimisant la variance entre les pixels clairs et foncés, offrant ainsi une bien meilleure adaptabilité aux variations de luminosité de l'image.

De plus, Baptiste a utilisé la librairie **MagickWand** afin d'effectuer certaines retouches sur l'image. Cet outil permet de sélectionner et de modifier des zones précises de l'image en fonction de la similarité des pixels, ce qui facilite le nettoyage et l'amélioration du rendu final.

Détection des Zones (Gaël)

Pour cette étape, l'objectif était de réduire la résolution d'une image binaire, c'est-à-dire ne contenant que des 0 et des 1 (0 représentant les pixels blancs et 1 les pixels noirs) afin de faciliter la détection des zones contenant des informations significatives, comme la grille ou la liste de mots. Le programme mentionné ci-dessous, a été implémenté, de même que l'était le solver, en langage Python, et sera traduit en langage C ultérieurement. L'algorithme utilisé repose sur un partitionnement de la matrice **matrix** de l'image en une grille **grid** de dimensions $n \times n$. Chaque case de cette grille correspond à un bloc de pixels de l'image d'origine, et sa valeur est déterminée en fonction de la proportion de pixels noirs qu'il contient en suivant la formule suivante :

$$\text{grid}[i][j] = \begin{cases} 1, & \text{si } \frac{\text{Nombre de Pixels Noirs}}{\text{Nombre Total de Pixels}} \geq \text{seuil} \\ 0, & \text{sinon} \end{cases}$$

On notera que :

- $\text{seuil} = 0.0 \implies$ Au moins un pixel noir suffit pour considérer le bloc comme noir
- $\text{seuil} = 0.5 \implies$ Une majorité de pixels noirs suffit pour considérer le bloc comme noir
- $\text{seuil} = 1.0 \implies$ Ne considérera jamais le bloc comme noir

Le découpage s'adapte automatiquement si la dimension de l'image n'est pas un multiple exact de n , en répartissant les pixels « restants » sur les premières lignes et colonnes de la grille. L'algorithme renvoie finalement une nouvelle matrice $n \times n$ représentant l'image réduite, où chaque cellule indique la présence ou l'absence d'une zone noire significative. Cette méthode, appelée **downscale**, permet de simplifier le traitement ultérieur pour la détection de zones, tout en conservant les informations structurantes de l'image.

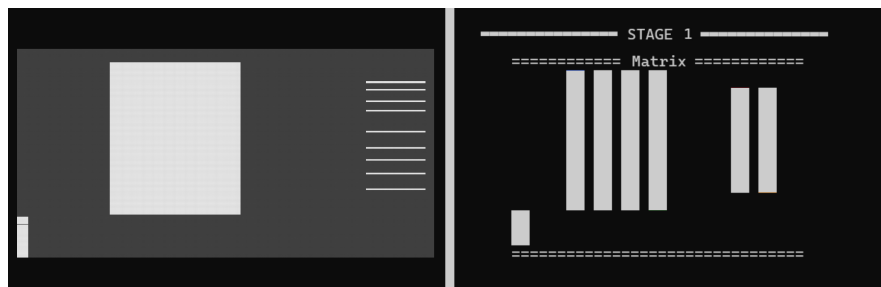


Figure 2.3: Exemple de *Downscale* d'une Image de 600×400 en une grille de 10×10 avec $\text{seuil} = 1\%$

Réseau de Neurones (Ethan)

Cette semaine, Ethan a poursuivi ses recherches sur le fonctionnement des réseaux de neurones et à la compréhension des différents types de neurones utilisés dans les architectures classiques. Il a ensuite implémenté des neurones à fonction d'activation sigmoïde, permettant d'introduire une non-linéarité dans le modèle et de rendre possible l'apprentissage de relations complexes entre les entrées et les sorties. Cette étape a constitué une base essentielle pour la mise en place d'un futur réseau de neurones plus complet.

Solver (Ethan, Gaël)

Cette semaine, Ethan a apporté plusieurs améliorations au programme du solver, notamment en ajoutant une fonction permettant de lire une matrice à partir d'un fichier texte. Par la suite, Gaël et Ethan ont intégré cette fonction dans le code de la procédure `main()` du solver, afin de permettre l'exécution de l'exécutable avec les paramètres appropriés.

Création de l'UI sur glade (Thibault)

Avant de créer l'UI il a fallu définir quels étaient les contrôles essentiels à ajouter à l'UI. Ces éléments sont :

- Un bouton pour charger une image
- Un bouton pour charger un modèle
- Un bouton pour lancer la résolution de grille

Il a également fallu choisir comment les disposer. Pour cela nous avons opté pour une barre latérale à gauche contenant tous les paramètres ainsi que les boutons précédemment cités et à droite l'image en cours d'utilisation. L'image par défaut est le logo du groupe, cette solution permet grâce aux paramètres d'extension de la fenêtre, de conserver une taille de fenêtre appropriée et de montrer à l'utilisateur où va se situer le "viewport".

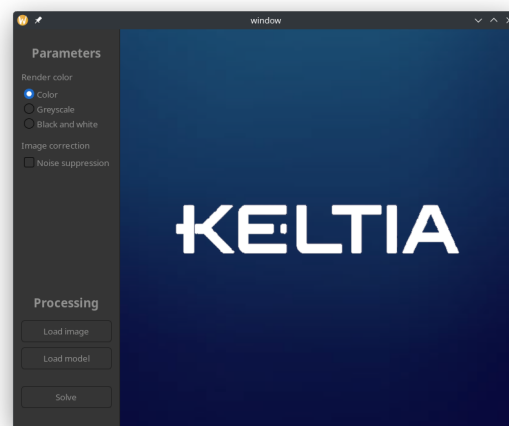


Figure 2.4: Aperçu de l'interface utilisateur

À FAIRE :

- Rotation d'Image (Baptiste) : Faire des recherches sur l'implémentation de la rotation.
- Détection des Zones (Gaël) : Continuer l'algorithme pour qu'il détecte deux zones dans une grille réduite.
- Réseau de Neurones (Ethan) : Faire des recherches sur les méthodes d'apprentissages.
- UI (Thibault) : Code behind et débogage GTK

2.4 Semaine du 13 octobre 2025

Rotation d'Image (Baptiste)

Cette semaine, Baptiste s'est consacré à l'implémentation de la rotation manuelle de l'image en fonction d'un angle donné. Il a de nouveau utilisé la librairie **MagickWand** pour réaliser des transformations supplémentaires sur les images concernant les rotations manuelles. Cette fonctionnalité permet de spécifier précisément l'angle de rotation souhaité (en degrés) et d'appliquer la transformation directement à l'image. L'utilisation de **MagickWand** s'est donc révélée pratique pour effectuer ces opérations de manipulation d'image grâce aux fonctions intégrées de la bibliothèque, sans nécessiter d'implémentation complexe supplémentaire.

```
#include <MagickWand/MagickWand.h>
#include <stdio.h>
#include <stdlib.h>

MagickWand *rotate_image(const MagickWand *input_wand, double angle)
{
    MagickWand *rotated_wand = NULL;
    MagickBooleanType status;
    PixelWand *background = NULL;

    // Clone the input image
    rotated_wand = CloneMagickWand(input_wand);
    if (rotated_wand == NULL)
    {
        fprintf(stderr, "Error: unable to clone the input MagickWand.\n");
        return NULL;
    }

    // Define the background color (white)
    background = NewPixelWand();
    PixelSetColor(background, "white");

    // perform the rotation (only 3 arguments)
    status = MagickRotateImage(rotated_wand, background, angle);
    if (status == MagickFalse)
    {
        fprintf(stderr, "Error: failed to rotate the image.\n");
        if (rotated_wand) rotated_wand = DestroyMagickWand(rotated_wand);
        if (background) background = DestroyPixelWand(background);
        return NULL;
    }

    // cleanup
    if (background) background = DestroyPixelWand(background);

    return rotated_wand;
}
```

Figure 2.5: Code de la fonction `rotate_image()`

Réseau de Neurones (Ethan)

Cette semaine, Ethan s'est d'abord concentré sur la recherche théorique sur les réseaux de neurones et sur les méthodes d'apprentissage associées. L'objectif principal était de consolider la compréhension des fondements nécessaires à l'implémentation d'un réseau de neurones dans le cadre du projet. Ces connaissances serviront de base pour le développement futur du module d'intelligence artificielle du projet.

Détection des Zones (Gaël)

Cette semaine, la tâche de Gaël consiste à identifier les deux zones principales dans une image binaire, après application du partitionnement (**downscale**). L'algorithme prend en entrée une matrice où chaque case vaut 0 (blanc) ou 1 (noir), et retourne les coordonnées des deux plus grands rectangles de pixels noirs. Le principe repose sur une parcours en profondeur (DFS) de la matrice :

1. Chaque cellule noire non visitée initialise une pile qui permet d'explorer toutes les cellules connectées à celle-ci (voisinage 4-connecté : haut, bas, gauche, droite).⁵
2. Pendant l'exploration, l'algorithme suit les coordonnées minimales et maximales atteintes afin de délimiter un rectangle englobant tous les pixels noirs connectés.
3. Une fois tous les pixels d'une zone visités, le rectangle est ajouté à une liste.
4. Après avoir identifié tous les rectangles présents dans la matrice, l'algorithme calcule l'aire de chacun et retient uniquement les deux plus grands. Les coordonnées retournées correspondent aux coins supérieur gauche et inférieur droit de chaque rectangle.

Cette méthode permet de détecter efficacement les deux zones importantes dans une image, tout en éliminant les petits regroupements de pixels isolés qui ne correspondent pas aux zones d'intérêt principales.

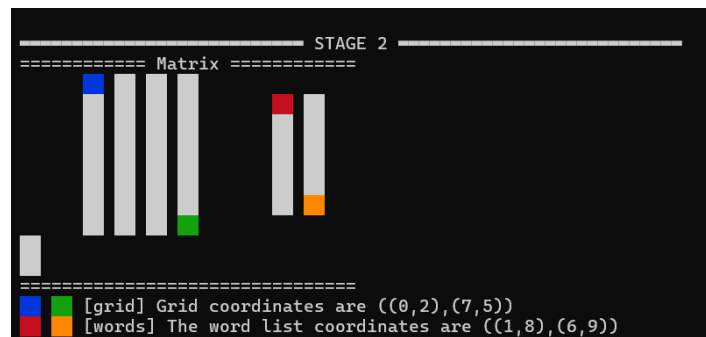


Figure 2.6: Exemple d'Identification des deux Zones

Gestion de Projet (Ethan)

En parallèle, un travail de restructuration du dépôt Git a été entrepris. L'objectif était de rendre le projet plus modulaire, plus facile à compiler et mieux organisé pour accueillir de nouveaux modules (notamment le réseau de neurones et les outils de traitement d'image). Cette tâche a impliqué une révision de l'arborescence des répertoires, une uniformisation des chemins d'inclusion, ainsi qu'une mise à jour des scripts de compilation (**Makefile**). Plusieurs difficultés ont été rencontrées lors de cette étape, notamment des conflits Git complexes dus à des divergences entre branches. Leur résolution a nécessité une intervention minutieuse pour préserver l'intégrité du code et assurer la compatibilité entre les différents modules.

À FAIRE :

- Suppression du Bruit (Baptiste) : Écrire le script permettant de supprimer le bruit sur une image.
- Détection des Zones (Gaël) : Terminer l'algorithme et l'implémenter en langage C.

⁵Cette méthode permet d'assurer la détection d'une zone même si elle contient des trous (coins manquants, espacement entre les lettres, ...). Cela rajoute de la fiabilité à l'algorithme dans des cas non-triviaux.

2.5 Semaine du 20 octobre 2025

Suppression du Bruit (Baptiste)

Ce script a pour objectif de supprimer le « bruit » présent sur une image à l'aide de fonctions intégrées à la librairie MagickWand. Le principe repose sur l'application d'un filtre médian sur l'image, qui consiste à remplacer la valeur de chaque pixel par la médiane des intensités des pixels situés dans un voisinage de taille $radius \times radius$. Cette méthode permet de réduire efficacement les imperfections tout en préservant les contours et les détails importants de l'image. Le rayon du voisinage ($radius$) est déterminé automatiquement par le programme, ce qui assure une adaptation du filtrage en fonction des caractéristiques de l'image traitée. L'aboutissement de ce script permettra de terminer celui de la détection des zones dont l'implémentation par Gaël est pratiquement achevée.

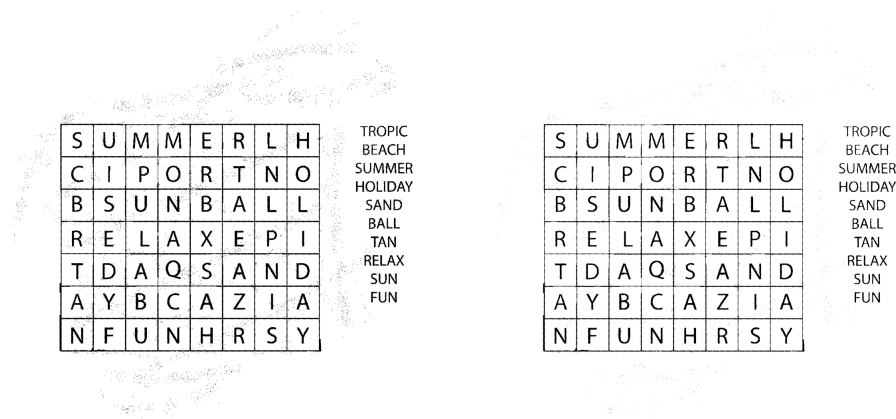


Figure 2.7: Exemple d'Application de l'Algorithme de Suppression du Bruit (Avant / Après)

Détection des Zones (Gaël)

Cette semaine, Gaël a terminé l'algorithme de détection des zones. La fonction principale prend en entrée une matrice de 0 et de 1, le nombre de subdivisions souhaité n pour la grille, ainsi qu'un `seuil` permettant de déterminer si une case de la grille doit être considérée comme noire. Le programme effectue successivement les étapes suivantes :

1. Partitionnement (`downscale`) de la matrice initiale en une grille $n \times n$. Chaque case représentant un bloc de pixels et valant 1 si la proportion de pixels noirs dépasse le seuil.
2. Détection des rectangles dans la grille réduite, pour identifier les deux zones principales sélectionnées en mesurant leurs aires respectives.
3. Conversion des coordonnées pour les reporter sur la matrice originale⁶, afin d'obtenir les limites exactes des zones détectées.

Cet algorithme a été entièrement réécrit en langage C, en conservant la même logique et les mêmes étapes que la version Python, tout en bénéficiant des performances accrues du C pour le traitement d'images de grande taille et pour les applications temps réel. Il permet ainsi d'obtenir automatiquement les coordonnées des zones d'intérêt et de préparer les données pour les étapes suivantes du programme. Un programme de tests a également été implémenté en langage C afin de servir de démonstration visuelle lors de la soutenance, affichant graphiquement dans le terminal des petites grilles modélisées à l'aide d'emojis de couleur, pour offrir une expérience visuelle illustrant l'algorithme décrit dans ce présent rapport.

⁶On admet que puisque la `grid` est une version minimisée de la matrice `matrix`, les coordonnées de la grille dans la matrice sont proportionnelles à la largeur et hauteur d'un bloc de la matrice correspondant à une case de `grid`

Réseau de Neurones (Ethan)

Au cours de cette semaine, Ethan a approfondi ses connaissances sur le fonctionnement de la rétropropagation dans les réseaux de neurones artificiels. Ce mécanisme repose sur le calcul d'une fonction de coût, qui mesure l'écart entre les sorties prédites par le modèle et les valeurs attendues. L'objectif de la rétropropagation est de minimiser cette fonction de coût en ajustant progressivement les poids du réseau. Pour cela, l'algorithme applique la dérivée de la fonction de coût par rapport à chaque poids, grâce à la règle de la chaîne, permettant une propagation de l'erreur du neurone de sortie vers les couches précédentes. Ce processus est essentiel pour l'apprentissage supervisé, car il guide le modèle vers une amélioration de ses performances à chaque itération. Les formules de modification des poids et des biais utilisées sont les suivantes :

$$w_k \rightarrow w'_k = w_k - \frac{\eta}{m} \sum_j \frac{\partial C_{X_j}}{\partial w_k}$$
$$b_l \rightarrow b'_l = b_l - \frac{\eta}{m} \sum_j \frac{\partial C_{X_j}}{\partial b_l}$$

2.5.1 Code behind et débogage GTK

Après la phase de création, il a fallu passer à la partie code. Le fichier `src/window/window.c` contiendra donc les fonction essentielles à l'affichage et à l'interaction avec l'application. Nous avons créé dans ce fichier, un événement permettant de faire en sorte que la fermeture de la fenêtre principale entraîne aussi la fermeture de l'application. Nous avons fait le choix de ne pas connecter tout de suite les composants aux fonctions de notre application car celle-ci sont encore en modification en risque de changer. Ainsi cela évite de tout casser lors de la modification d'une fonction. Il a également fallu créer une `g_ressource` pour le logo de l'entreprise afin que celui-ci soit inclus dans le binaire du programme ce qui permet de le rendre portable et non dépendant d'un chemin de fichier absolu. Cependant nous avons remarqué un bug provenant de GTK, en effet, utiliser l'address sanitizer avec GTK est inutile car par défaut, la bibliothèque alloue de la mémoire sans la libérer avant la fin de l'exécution causant grand nombre de message de memory leak non provoqué par notre code. Nous avons donc désactivé address sanitizer pour les tests de la fenêtre.

À FAIRE :

- Documents de Projet (Gaël) : Rédaction des documents⁷ supports du projet pour l'équipe.
- Débogage (Baptiste, Gaël) : Vérification de la compatibilité des algorithmes de `greyscale()` et `remove_noise()` avec les algorithmes de `detect_zones()`
- Algo détection de caractères (Thibault)

⁷Documents : Rapport (*Ce présent document*), Plan de Soutenance

2.6 Semaine du 27 octobre 2025

Débogage (Baptiste)

Cette semaine, Baptiste a consacré du temps au débogage des scripts développés précédemment. L'objectif principal était de corriger les erreurs rencontrées lors de l'exécution des programmes, d'améliorer la robustesse des fonctions et de garantir un comportement fiable sur différents types d'images pour la soutenance. Cette phase de débogage a permis d'assurer que les algorithmes de conversion en niveaux de gris, de suppression du bruit et de rotation d'images fonctionnent correctement et de manière cohérente.

Débogage (Gaël)

Gaël a effectué cette semaine une relecture complète des algorithmes du solveur et de la détection des zones. Pour le solveur, il s'agissait principalement de vérifier la cohérence des indices dans les grilles, de s'assurer que les directions étaient correctement explorées et que les coordonnées retournées correspondaient bien aux conventions attendues. Pour la détection des zones, le débogage a porté sur le bon fonctionnement du `downscale`, la répartition correcte des pixels restants dans la grille, et essentiellement sur l'identification des deux rectangles principaux via l'algorithme `find_rectangles()`. En effet, des doutes subsistaient sur la fiabilité de cet algorithme dans des conditions réelles, qui, lors du développement, n'avait été testé que sur des cas parfaits, ou triviaux, attendant l'aboutissement du travail de Baptiste sur la suppression du bruit pour le tester dans les bonnes conditions. Cela a permis de s'assurer que les coordonnées renvoyées correspondaient aux zones réelles de l'image, et que les conversions entre la grille réduite et la matrice originale étaient exactes.

Réseau de Neurones (Ethan)

Cette semaine, Ethan a réalisé l'implémentation complète de l'algorithme de rétropropagation (`back propagation`) et la mise en place du processus d'entraînement des réseaux de neurones. L'objectif principal était de concevoir un système générique et évolutif, capable de s'adapter à n'importe quelle architecture de réseau et à tout type de jeu de données, sans dépendre d'un modèle fixe. L'implémentation repose sur une structure modulaire permettant de définir dynamiquement le nombre de couches, le nombre de neurones par couche, ainsi que les fonctions d'activation associées. Afin de valider le bon fonctionnement du mécanisme d'apprentissage, un premier exemple concret a été réalisé : l'entraînement d'un petit réseau de neurones pour simuler le comportement de la porte logique NXOR, problème classique démontrant la capacité d'un réseau multicouche à modéliser des relations non linéaires.

2.6.1 Détection caractères (Thibault)

Pour la détection de caractère nous avons utilisé un algorithme de remplissage par diffusion (`flood fill`). Le principe est simple, on itère dans la grille pour trouver un pixel noir. Puis on se déplace grace à une pile de pixel noir en pixel noir jusqu'à avoir détecté les pixel noirs adjacents. Ainsi on peut définir une zone du pixel le plus en haut à gauche au pixel le plus en bas à droite comme contenant un caractère. On enregistre ses coordonnées dans un tableau. Plus limité les erreurs, on admet qu'un caractère ne pas pas faire moins de 4 pixels, ainsi on réduit le risque de détecter le bruit résiduel. Parfois la bordure de la grille est aussi détecté. Pour la supprimer, on compare sa taille aux autre éléments du tableau et on admet qu'un caractère ne peut pas être 2 fois plus grand que la moyenne, si c'est le cas on le supprime.

Gestion de Projet (Ethan)

En parallèle du réseau de neurones, une refactorisation complète de la base de code liée aux réseaux de neurones a été effectuée. Cette restructuration a notamment permis de modulariser les différentes méthodes d'activation afin de faciliter leur utilisation et leur extension, d'intégrer directement la fonction d'activation à l'intérieur de chaque couche, assurant une meilleure encapsulation et une plus grande

cohérence dans le flux de calcul et de rendre le système plus flexible, permettant désormais de changer la fonction d'activation d'une couche sans modifier la structure globale du réseau. Cette itération du développement consolide donc la partie “apprentissage” du projet, tout en préparant le terrain pour des tests sur des réseaux plus complexes et des ensembles de données réels.

Documents de Projet (Gaël)

Cette semaine, Gaël a rédigé le rapport de projet en $\text{\LaTeX}$. Il s'est appliqué à structurer le document de manière claire et cohérente, en respectant les conventions et le format exigé. L'utilisation de $\text{\LaTeX}$ a permis d'obtenir une mise en page professionnelle, facilitant ainsi la lecture et la compréhension du contenu technique du rapport.

Part III

Bibliographie et Sources

Chapter 3

Bibliographie et Sources

3.1 Technique

3.1.1 Structures de Données

- Stacks
- Neural Networks

3.1.2 Algorithmes

- Algorithme de N. Otsu¹, 1979.
- Tutoriel vidéo : *Backpropagation Algorithm Explained*, 3Blue1Brown (YouTube), 2021.
- Tutoriel : *Word Search Solver Implementation in C*, GeeksForGeeks, 2023.

3.1.3 Tutoriels et Ressources Vidéo

- Vidéo : *Otsu Thresholding Algorithm Tutorial*, DigitalSreeni (YouTube), 2022.
- Vidéo : *Makefile Basics for C Projects*, Jacob Sorber (YouTube), 2023.
- Documentation : *Managing Git Hooks for CI/CD*, Atlassian Developer Docs, consultée en 2025.
- Vidéo : *Implementing Stacks and DFS in C*, CodeVault (YouTube), 2023.

3.1.4 Bibliothèques

- Documentation : *MagickWand API for C*, version 7.1.1, consultée en 2025.

¹A Threshold Selection Method from Gray-Level Histograms

Conclusion

Conclusion

Le projet **OCR-KeltIA** constitue une expérience riche et formatrice, tant sur le plan technique que méthodologique. Il nous permet d'aborder des thématiques complexes telles que la **reconnaissance optique de caractères**, le **traitement d'images** et l'implémentation d'un **réseau de neurones artificiels** en langage C. L'objectif principal étant concevoir une application capable d'analyser et de résoudre automatiquement une grille de mots cachés à partir d'une image, ce projet a d'abord nécessité une approche rigoureuse, une conception réfléchie et une coordination entre les membres du groupe.

Le développement en langage C nous a conduits à porter une attention particulière à la gestion de la mémoire, aux performances et à la structure du code, ce qui nous permet d'améliorer nos compétences dans ce langage. D'autre part, la mise en place de **Makefile**, de scripts d'automatisation et de *Git hooks* a permis d'assurer la cohérence du dépôt et la standardisation du formatage, facilitant ainsi le travail collaboratif. L'usage d'outils tels que Vim, **VSCoDe** et le compilateur **GCC** s'est révélé essentiel pour maintenir un environnement de développement stable et efficace.

Sur le plan algorithmique, la conception d'un réseau de neurones fonctionnel, capable d'apprendre à reconnaître des caractères à partir d'images, a constitué un défi majeur. Nous avons dû expérimenter différentes architectures et fonctions d'activation, tout en assurant la compatibilité avec la structure modulaire du projet. Le système final, combinant la reconnaissance, illustre la proximité entre les domaines du traitement de données visuelles et de l'intelligence artificielle.

En dehors des aspects techniques, ce projet nous a permis de développer des compétences en **gestion de projet**, en **travail d'équipe** et en **résolution de problèmes**. La répartition des tâches, la communication continue avec des réunions régulières et l'utilisation des branches Git pour l'intégration progressive des fonctionnalités ont été des facteurs clés de réussite. Chaque membre a pu apporter sa contribution spécifique tout en participant à une vision commune, celle de produire un logiciel complet et cohérent avec les consignes imposées.

En conclusion, **OCR KeltIA** représente non seulement l'aboutissement d'un travail collectif rigoureux, mais aussi une application concrète des enseignements théoriques reçus. Il illustre notre capacité à concevoir un système complet, du bas niveau jusqu'à l'intelligence artificielle, tout en respectant les contraintes académiques et techniques imposées.